PORTFOLIO · 2026.11

장하은

AI Service Developer

AI 결과를 그대로 쓰지 않고, 검증해서 서비스로 만듭니다

검증하는 AI · 데이터로 판단 · 끝까지 만들고 다시 고치기

Email jhe290609@gmail.com **GitHub** github.com/haeun910 **Portfolio** haeun910.github.io

PROJECTS

01 **Veil AI**
LLM 앞단 민감정보 탐지 모델

02 **AI 채용 평가**
LLM 이력서 평가 서버 · 신뢰도 점검

03 **BARO**
공공데이터 민원 안내 서비스

04 **경남 보행안전**
보행 위험 후보지 분석 플랫폼

05 **Moa**
할 일 · 일정 관리 웹앱 (PWA)

ABOUT ME

AI 결과를 그대로 쓰지 않고, 검증해서 서비스로 만드는 AI 서비스 개발자 장하은입니다.

01

검증하는 AI

AI 결과를 그대로 내보내지 않고 확인하는 장치를 만듭니다. LLM에 보내기 전 민감정보를 찾아내는 모델을 학습하고, 같은 이력서를 바꿔 가며 다시 채점하는 점검 도구를 만들고, 서버와 화면이 점수 방향을 다르게 읽던 오류를 찾아 고쳤습니다 (2026.10 고도화).

→ 01 Veil AI · 02 AI 채용 평가

02

데이터로 판단

데이터를 직접 만들고, 나눠서 비교한 뒤 결정합니다. 학습 문장 **49,576개**로 모델 **4종**을 같은 조건에서 비교했고, 과거 자료로 고른 후보지를 이후 실제 사고로 채점했습니다.

→ 01 Veil AI · 04 경남 보행안전

03

끝까지 만들고 다시 고치기

기획부터 배포까지 혼자 끝내고, 배포한 뒤에도 다시 열어 고칩니다. 1인 개발로 배포한 민원 안내 서비스의 버스 노선 반복 조회를 서버 캐시로 **3.18초**에서 **0.30초**로 줄였습니다 (2026.10 고도화).

→ 03 BARO · 05 Moa

CONTENTS

About Me 02
Skill 03
수상 · 자격 · 학력 04
01 Veil AI 05
02 AI 채용 평가 07
03 BARO 09
04 경남 보행안전 11
05 Moa 13
그 밖의 프로젝트 14
Contact 15

SKILL

기술 스택

AI · LLM

Claude API | GPT-4o | KLUE-RoBERTa | PyTorch | Transformers | Gemini API | YOLO | Raspberry Pi

Data

Python | pandas | numpy | scikit-learn | 공공데이터 API | PyPDF2 | python-docx | Matplotlib | Jupyter

Backend

FastAPI | Supabase | PostgreSQL | Flask | Node.js | Vercel Serverless | ReportLab | Streamlit | Docker | pytest

Frontend · Deploy

React | Vite | HTML | CSS | JavaScript | Leaflet | Vercel | TypeScript | Tailwind CSS | PWA | GitHub | GitHub Pages

AWARDS & EDUCATION

수상 · 자격 · 학력

수상

2026.08 — **빅데이터 우리동네 공모전** 최우수상
경남형 보행위험 후보지 자동선정 플랫폼 → 04 경남 보행안전

2026.02 — **생성형 AI+X 아이디어 경진대회** 최우수상 (2등)
AI 신뢰도 기반 다중 에이전트 채용 평가 → 02 AI 채용 평가

2026.01 — **TRAITHON 2025** 투명성상
낚시성 기사 탐지 AI의 신뢰성 평가

2025.08 — **IoT GPT Creative Challenge** 최우수상
진주시 전월세 분석 GPTs

2025.07 — **IoT Invent-On** 최우수상
반려견 분리불안 완화 AIoT

자격증

2026.06 — **정보처리기사**
필기 합격

2025.11 — **ADsP 데이터분석 준전문가**

2025.10 — **DAsP 데이터아키텍처 준전문가**

학력 · 연구

2027.02 — **경상국립대학교 AI정보공학과**
졸업 예정

학부연구생 — **다중에이전트시스템연구실**
2022.05 – 2023.02 · 2025.03 – 2027.02

01

캡스톤디자인 · 생성형 AI 정보 유출 방지

Veil AI LLM에 보내기 전에 민감정보부터 가립니다

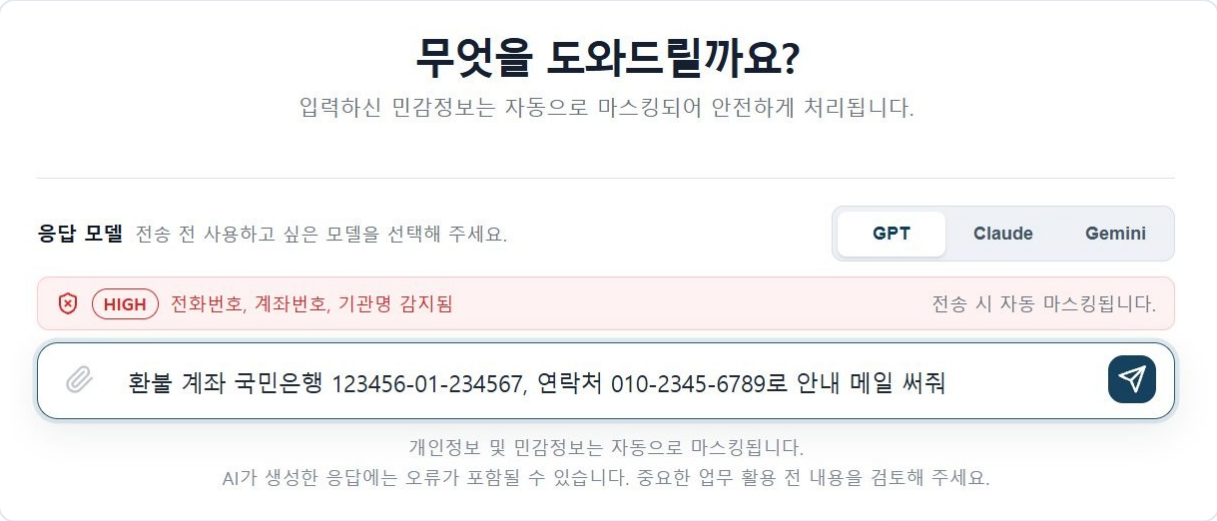
2026.03 ~ 2026.06 · 팀 4인

담당 진행 총괄 · 학습 데이터 구축 · 모델 파인튜닝 · 랜딩 · 로그인

PyTorch · Transformers · React

↗ 서비스 ↗ GitHub

#NER파인튜닝 #데이터셋구축 #LLM보안



채팅 입력 화면. 보내기 전 위험도와 감지된 민감정보 표시



구조. 외부 LLM과 DB에는 가린 문장만 전달

문제

- 직원이 생성형 AI에 고객 정보 · 계좌번호 · 문서명을 그대로 입력
- 외부 LLM으로 간 문장은 회사가 확인 불가

해결 방법

01 1차 규칙 (정규식)
정해진 형식 찾기 · 전화 · 계좌번호

02 2차 AI 모델 (NER)
문맥으로 이름 · 기관명 · 문서명 찾기

03 3차 위험도
높음 · 보통 · 낮음 3단계 판정

04 가린 문장만 전송
[계좌번호] · [전화번호]로 바꿔 전송

내가 한 일

- **학습 데이터 구축** 학습 문장 **49,576개** 설계 · 구축 (탐지 항목 **8종**)
- **모델 파인튜닝** 한국어 AI 모델 **4종** 비교 → KLUE-RoBERTa 적용
- **진행 총괄** 주제 선정, 일정 · 역할 관리, 중간 발표
- **랜딩 · 로그인** 화면 직접 구현, 잘못 탐지 · 원문 저장 문제 수정

모델	**KLUE-RoBERTa NER** F1 **96.14%** (미학습 **4,996문장** · 항목 단위) · 평균 **0.066초** (CPU 서버)
팀 스택	FastAPI · Supabase · Vercel

01 학습 데이터를 직접 만든 이유

평가 기준 같은 출처 · 생성 규칙으로 만든 학습 미사용 문장 4,996개 · 항목 단위

데이터 · 출처

공개 데이터에 없는 항목은 규칙으로 생성

- 탐지를 외부 LLM에 맡기면 그 자체가 유출, 로컬 LLM은 GPU 비용이 큼 → 작은 모델을 직접 학습
- 공개 학습 데이터에는 사내 문서명 · 프로젝트명 · 비밀번호 라벨이 없음
- 이름 · 기관명은 KLUE-NER 원본, 주소 · 금융은 원본과 규칙 생성 절반씩

데이터 · 구성

실제 업무 입력처럼 섞어서 구성

- 민감정보 **1개** 또는 없음 39,624 · 여러 개 4,952 · 여러 문장 5,000
- 민감정보처럼 보이지만 아닌 문장을 넣어 괜히 가리지 않는지 확인

모델

같은 조건에서 4종을 비교한 뒤 채택

- 같은 데이터 · 분할로 학습, 검증 점수가 가장 높은 8 에폭(반복 횟수) 모델 사용
- 실제 업무 문장으로 재현 → 수정 → 재확인, 규칙 자동 테스트 **32건** 통과

같은 데이터로 비교한 한국어 모델 4종

모델	정밀도	재현율	F1
KLUE-RoBERTa 적용	**95.59%**	**96.70%**	**96.14%**
KoELECTRA-base	95.21%	96.08%	95.64%
KoELECTRA-NER	94.58%	95.70%	95.14%
KcBERT-base	87.27%	91.04%	89.12%

정밀도: 가린 것 중 진짜 민감정보 비율 · 재현율: 민감정보 중 찾은 비율 · F1: 둘의 종합

KLUE-RoBERTa를 적용했습니다. 정밀도 · 재현율 · F1 모두 **4개** 모델 중 가장 높았습니다.

라벨별 F1 · KLUE-RoBERTa · 결과가 기록된 7종

비밀번호	금융	프로젝트명	주소	이름	기관명	문서명
100%	99.78%	97.55%	95.99%	94.62%	93.46%	91.86%

다중 에이전트 · LLM 평가 신뢰성

02 AI 채용 평가 점수가 아니라 신뢰를 평가합니다

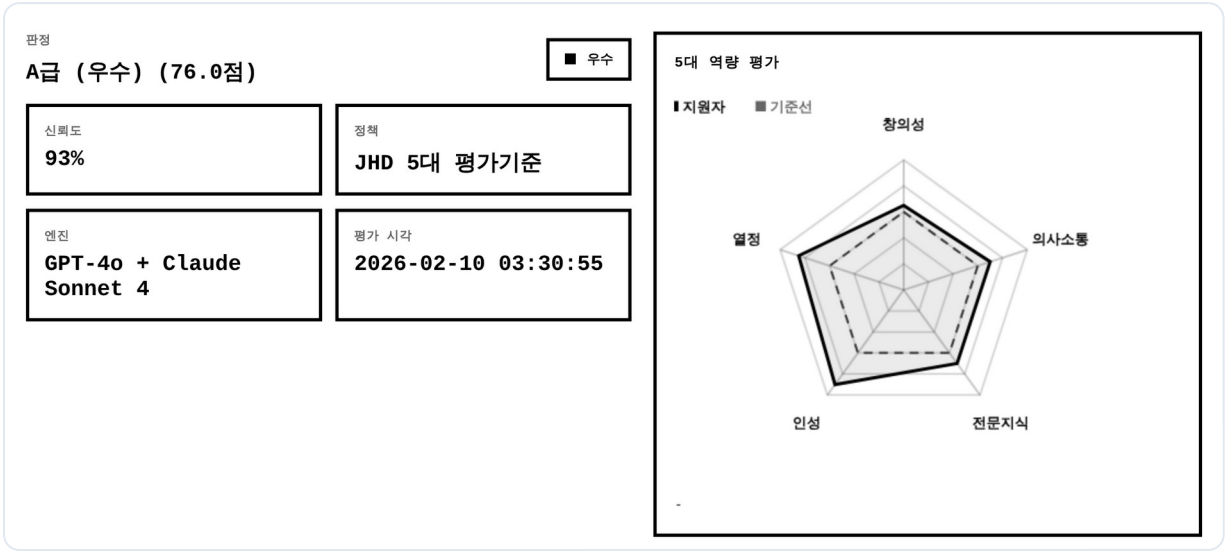
2025.12 ~ 2026.02 · 2026.10 고도화 · 팀 2인 · 최우수상

담당 기획 · 신뢰성 감사 프로토타입(1~3차) · 에이전트 흐름 설계 · API 서버 · 발표 자료 제작

Python · Flask · GPT-4o · Claude

↗ GitHub

#신뢰성감사 #멀티에이전트 #FlaskAPI #PDF리포트



결과 화면. A급 76.0점 · 93% = 종합 신뢰도 92.5 (최종 점수와 다른 지표)



설계. 신뢰성 낮으면 보류 → 사람이 재검토

문제

- **불투명성** 점수의 이유를 알 수 없음
- **불안정성** 형식 · 키워드에 따라 점수가 흔들릴 수 있음
- **검증 부재** 믿어도 될지 모름

시스템 구성 · 팀 2인

01 평가 에이전트
5대 기준 하나씩 채점

02 설명 에이전트
등급 · 강점 · 보완점

03 검증 에이전트
Claude가 결과로 재채점

04 PDF 리포트
같은 결과를 A4로

내가 한 일

- **신뢰성 감사 프로토타입(1~3차)** 다시 채점하는 도구 → 직무별 가중치 → 서버 분리
- **에이전트 흐름 설계** 평가 · 설명
- **Flask API 서버 · PDF** 파일 처리 · 응답 · 한글 A4
- **계산 오류 수정 (2026.10 고도화)** 종합 92.5 · 최종 24.6 모순에서 점수 방향 차이를 찾아 수정 → 73.8

스택 **GPT-4o** 평가 · 설명 · **Claude Sonnet 4** 검증 · ReportLab PDF

02 같은 이력서를 바꿔 가며 다시 채점한 이유

기준 1~3차 프로토타입 · API 서버

1차 · 감사

점수가 흔들리는 정도로 신뢰성을 판정

- 형식 · 키워드만 바꿔도 점수가 달라지면 그 점수는 믿기 어려움
- 반복 · 형식 변형 · 키워드 반복 이력서를 만들어 다시 채점하고 변동폭을 계산
- 기준값을 넘으면 위험으로 판정하고 이유 문장을 함께 출력

2 · 3차 · 기업용

직무별 가중치와 판정 보류

- 직무마다 세 기준(직무 적합성 · 근거성 · 명확성)의 비중을 다르게
- 3차에서 서버를 분리하고, 규칙 기반 **6개** 축으로 채점

API 서버

이력서를 서버에 남기지 않음

- DB 없이 처리, 성공 · 실패 · 오류 모든 경우에 업로드 파일 삭제
- 오류 응답은 안내 문구만 전달해 내부 경로 · 코드 노출 제거

1차 판정 기준

측정	방법	주의	위험
반복 채점 점수 편차	같은 이력서 5번 채점	4점 ↑	**6점 ↑**
형식만 바꾼 점수 차	문단 · 글머리표 · 순서 바꿈	12점 ↑	**20점 ↑**
키워드로 오른 점수	키워드 1 · 3 · 5번	+8점 ↑	**+12점 ↑**

2차 직무별 가중치

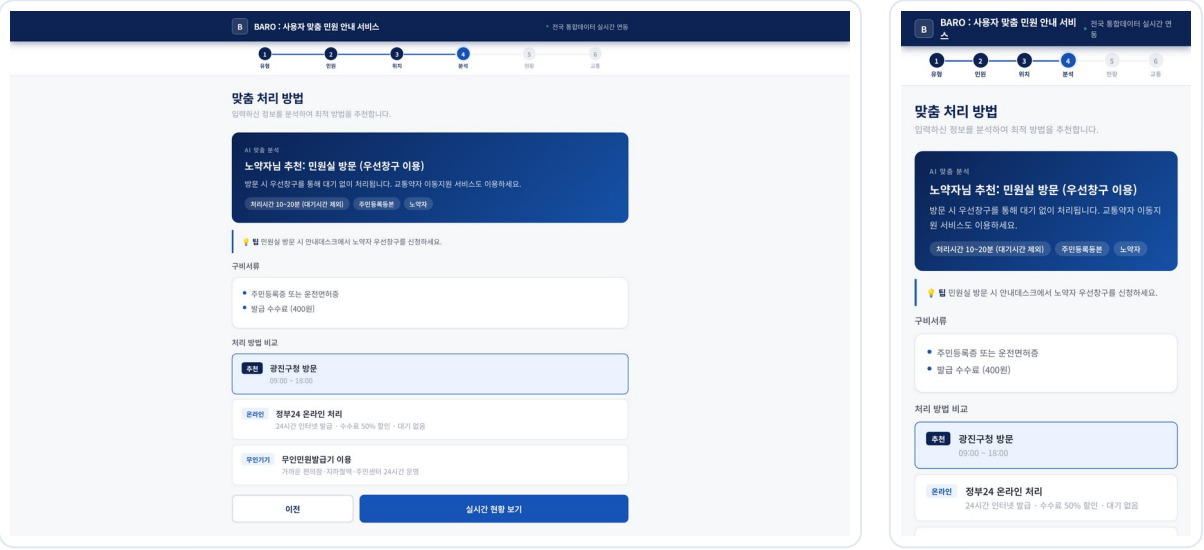
직무	직무 적합성	근거성	명확성
인사(HR)	40%	40%	20%
마케팅	45%	45%	10%
개발	45%	40%	15%

3차 프로토타입: 신뢰성 지표가 하나라도 60 미만이면 불합격선은 넘었더라도 합격 대신 보류했습니다.

03 BARO 헛걸음 없이, 방문 전에 처리 방법부터

2026.04 · 2026.10 고도화 · 1인 개발
담당 기획 · 데이터 조사 · 프론트 · 서버 · 배포
Vanilla JS · Leaflet · Vercel Serverless · Claude API
↗ 서비스 ↗ GitHub

#공공데이터API #서버리스 #LLM추천 #반응형웹



AI 처리 방법 (PC · 모바일). 노약자 · 주민등록등본 선택 시 추천 방법 · 이유 · 구비서류



구조. 화면은 서버를 통해서만 공공데이터 · AI를 호출

문제

- 민원의 가장 큰 불편은 불필요한 방문과 예측할 수 없는 대기
- 고령층 · 장애인 · 영유아 동반 보호자에게는 헛걸음 한 번이 큰 부담

해결 방법

01 유형 · 민원 선택
방문자 6유형 · 주요 민원 10종

02 주변 민원실
GPS · 주소 → 거리순 목록 · 지도

03 AI 처리 방법 추천
방문 · 온라인 · 무인기기 · 구비서류

04 실시간 대기 · 교통
업무별 대기 30초 갱신 · 버스 노선

내가 한 일

- **기획 · 데이터 조사** 공공데이터 실시간 API **4종** + Claude API 연결
- **서버** 인증 키 보호, 기관마다 다른 응답 형식을 한 곳에서 통일
- **프론트** 6단계 반응형 화면, 지도 (Leaflet)
- **배포 · 성능 개선** 서버 캐시로 버스 반복 조회 **3.18초** → **0.30초** (2026.10 고도화)

AI	**Claude API** 사용자 유형별 처리 방법 추천 · **7개** 항목으로 답변
데이터	민원실 이용 현황 · 초정밀 버스 · 교통약자 이동지원 · 신호등

03 공공데이터를 서버 한 곳에서 거친 이유

측정 기준 배포 사이트에서 같은 조회를 반복해 잰 중앙값

보안

인증 키는 서버에만

- 브라우저는 다른 기관 데이터를 직접 못 부름(CORS 제한) → 서버가 대신 호출
- 인증 키 **2종**은 서버 설정에만 저장하고 화면 코드에는 두지 않음

데이터 · 2026.10 고도화

응답 형식은 서버 한 곳에서 통일

- 기관마다 다른 응답을 서버 공통 코드 하나로 정리
- 기관 응답 형식이 바뀌어도 화면 코드는 그대로
- 데이터가 없는 지역 · 민원실은 임의 값 대신 "정보 없음"으로 표시

저장 후 재사용 · 2026.10 고도화

보관 시간은 데이터마다 다르게

- 자주 안 바뀌는 민원실 · 버스는 1시간, 이동지원센터는 **10분**
- 계속 바뀌는 대기 인원은 **10초**만 보관해 최신성 유지
- 저장은 서버(캐시)와 사용자 가까운 서버(CDN) 두 곳

서버 공통 코드의 규칙 5가지

규칙	공공데이터 원본	화면이 받는 형식
① **성공 여부 확인**	성공 신호 "K0" · "00"	**둘 다 성공으로 인식**
② **목록 꺼내기**	기관마다 다른 응답 구조	**같은 목록 하나로**
③ **항목 이름 통일**	기관이 정한 약어 이름	**화면용 이름 (이름 · 주소)**
④ **표시 형식 변환**	운영 시작 "090000"	**09:00 ~ 18:00**
⑤ **지역 거르기**	행정구역 번호 "1121500000"	**앞 2자리 "11" → 서울**

AI 답변도 정해진 7개 항목으로 받습니다. 서버에서 형식을 확인한 뒤 화면 칸에 표시합니다. 형식이 틀리면 오류(502), 최대 **30초** 대기.

같은 조회를 반복할 때 걸린 시간 · 2026.10 고도화

조회 (반복 · 중앙값)	방법	수정 전 → 수정 후
버스 노선	서버 메모리 캐시	**3.18초 → 0.30초 (−90%)**
민원실	서버 저장분 재사용	**0.42초 → 0.22초 (−48%)**

04 경남 보행안전 사고 기록을 겹쳐 먼저 점검할 곳을 고릅니다

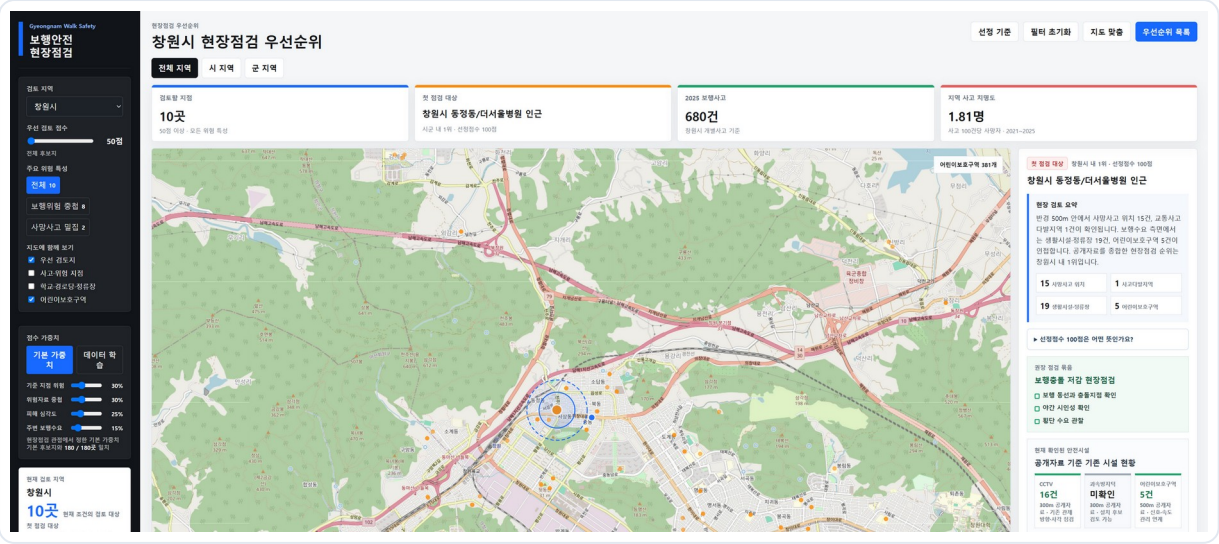
2026.07 ~ 2026.08 · 2026.10 고도화 · 팀 2인 · 최우수상

담당 데이터 수집 · 탐색 분석 · 위험 점수 · 검증 · 지도

Python · pandas · scikit-learn · Leaflet

↗ 플랫폼 ↗ GitHub

#공공데이터 #좌표분석 #위험점수설계 #백테스트



현장점검 플랫폼. 시군별 후보지 지도 · 선정 근거



구조. 결과 파일을 GitHub Pages로 공개

문제

- **인력 · 예산 18개** 시군 도로를 다 점검할 수 없음
- **선정 방식** 민원 · 경험에 의존해 위험 지점을 놓침
- **사고 수 기준** 사고 수는 인구를 따라가 큰 도시만 뽑힘

해결 방법

01 데이터 정리
위험자료 9종 → 위험 기록 7,174곳

02 위험 점수
지점 · 반경 500m 근거 4가지 → 시군별 10곳

03 백테스트
과거 자료로 고르고 이후 사고로 채점

04 웹 지도
비중을 바꾸면 180곳 바로 재선정

내가 한 일

- **데이터 수집 · 정리 · 탐색 분석 · 지도 플랫폼**
- **위험 점수 설계 · 검증** 2026.10 고도화, 경남 면적 **1.2%**에서 이후 사고 **25.1%** 포착 · 무작위 생활시설 약 **4배**, 무작위 위험 기록 약 **2.4배**

모델	**위험 점수** 근거 4가지 가중합 · 로지스틱 회귀(사고 예측 머신러닝) 비교
도구	pandas · scikit-learn · JavaScript · Leaflet

04 사고 수 대신 주변 근거로 고른 이유

검증 2021년까지 자료로 선정 → 2022–24년 실제 사고로 채점

탐색 분석

사고 수는 인구를 그대로 따라감

- 사고 수와 인구의 상관계수(1이면 완전히 같이 움직임) **0.978**: 사고 수로 고르면 큰 도시만
- 인구 대비로 보면 통영시 **6위 → 1위**
- 그래서 지점과 반경 **500m** 안 위험 근거 4가지를 겹쳐 보는 점수로 설계

선정

시군별 10곳, 후보 간 500m 간격

- 큰 도시에 몰리지 않게 시군마다 **10곳**씩 = **180곳**
- 간격 **500m** = 점검 반경: 한 후보가 다른 후보 반경 안에 들지 않게

백테스트 · 2026.10 고도화

과거 자료로 고르고, 이후 사고로 채점

- 2022–24년 사망사고 **468건** + 새로 지정된 사고다발지역 **89곳** = **557건**
- 간격 **500m**는 검증 기간 비교(120 · 250 · **500m** = 13.7 / 14.8 / **17.0%**)로 확정

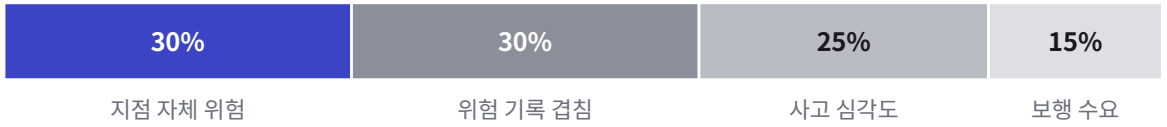
백테스트 · 2026.10 고도화 · 이후 실제 사고 557건

고르는 방식 (180곳 · 반경 500m)	이후 사고가 그 안에서 난 비율
위험 점수 · 내가 정한 비중	**25.1% (140건 = 사망사고 76 + 새 다발지역 64)**
위험 점수 · 머신러닝이 찾은 비중	27.3%
과거 사망사고가 많은 곳만	22.3%
무작위로 고른 위험 기록	10.5%
무작위로 고른 생활시설	5.9%

후보지 반경 합계 = 경남 면적 **1.2%**, 그 안에서 이후 사고 **4건** 중 **1건**

내가 정한 비중을 기본값으로 두었습니다. ±**0.10** 바꿔도 **23.2–26.9%**. 머신러닝 비중은 지점 자체 위험 0, 심각도 · 수요 **0.43** · **0.41**로 쏠려 '데이터 학습' 버튼으로 함께 두었습니다.

위험 점수 = 지점과 반경 500m 안 근거 4가지의 비중

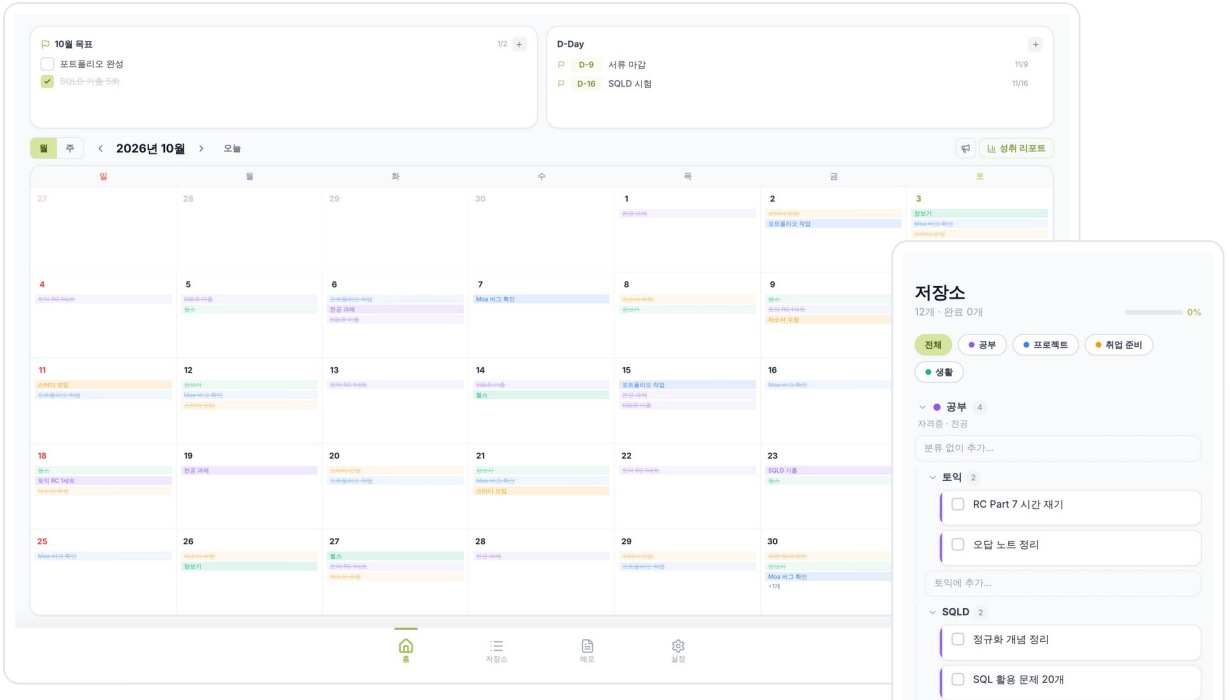


개인 프로젝트 · 개인 플래너 PWA

05 모아 (Moa) 날짜 없는 할 일은 저장소에, 할 날이 되면 오늘로

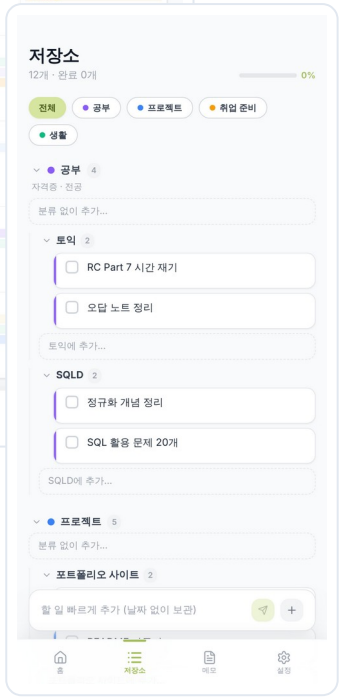
2026.06 ~ 개발 진행 중 · 1인 개발
담당 기획 · 디자인 · 프론트엔드 · DB 설계 · 배포
React 19 · TypeScript · Supabase · PWA
↗ 서비스 ↗ GitHub

#React #TypeScript #Supabase #PWA #실시간동기화



PC 홈. 목표 · D-Day · 월간 달력

반응형 · 설치형 코드 하나로 PC · 태블릿 · 모바일 화면에 맞춰 배치하고, 홈 화면에 앱처럼 설치 (PWA)



모바일 저장소. 카테고리별 보관

문제

- 날짜 없는 "언젠가 할 일"과 날짜가 정해진 일정을 따로 관리해 놓치는 일이 생김

주요 기능

01 저장소
카테고리 › 하위카테고리 › 할 일, 드래그로 이동

02 캘린더
월간 · 주간 보기, 반복 일정, 마감일

03 목표 · 리포트
월간 목표 · D-Day, 월 달성률 · 최근 14일 추이

04 동기화 · 설치
PC · 모바일 즉시 반영, 홈 화면에 앱처럼 설치

설계에서 신경 쓴 것

- **DB 단계 데이터 보호** RLS(행 단위 보안: DB가 로그인한 본인 데이터만 허용)
- **기기 간 동기화** Realtime(변경 전달)으로 다른 기기 화면에 바로 반영
- **v2.0 구조 전환** 3단 구조로 바꾸고 기존 데이터는 한 묶음(트랜잭션)으로 안전하게 이전
- **원인 찾기** 사용 중 홈으로 튕김 → 로그인 정보 자동 갱신이 원인, 실제 사용자 변경 때만 다시 불러옴

화면	React 19 · TypeScript · Vite · Tailwind CSS · PWA
서버	**Supabase** 로그인 · DB + RLS · Realtime · 트리거(DB 자동 실행) · Vercel 배포

MORE PROJECTS

그 밖의 프로젝트

06

Oreum

AI가 질문을 만들고, 고정 기준으로 비교하는 모의 면접

2025.10 · 팀 2인

담당 주제 선정 · 기획 · 백엔드 개발 · 발표 자료

Python · Streamlit · Gemini API

↗ GitHub

07

TRAITHON

낚시성 기사 탐지 AI가 틀리기 쉬운 기사로 신뢰성 평가 데이터 구축

2025.10 – 2026.01 · 팀 3인 · 투명성상

담당 데이터 분석 · 평가 데이터 구축 · 품질 검증

Python · pandas · scikit-learn

↗ GitHub

08

반려견 분리불안 완화 AIoT

1인 가구 반려견의 불안 행동을 카메라로 감지해 바로 달래주는 기기

2025.06 – 2025.07 · 팀 3인 · 최우수상

담당 팀장 · 시스템 통합 · Pi 추론 · 제어 로직 · 서버

Python · Raspberry Pi 5 · YOLOv11n · FastAPI

↗ GitHub

FundPilot	정책자금 추천 서비스: 신청 가능 여부(규칙) → 적합도 추천(유사도) → 리스크(이상 탐지) 3단 흐름 기획	2026.06 · 팀 · 기획 · 흐름 설계 · 검토	↗ GitHub
진주 전월세 GPTs	진주시 전월세 자료를 정리해 넣고 질문에 답하는 맞춤형 GPT	2025.08 · 최우수상	

CONTACT

읽어 주셔서 감사합니다.

Email

jhe290609@gmail.com

GitHub

github.com/haeun910

Portfolio

haeun910.github.io

프로젝트 링크

프로젝트	서비스	GitHub
01 Veil AI	↗ veil-ai-platform.vercel.app	↗ github.com/haeun910/leak_prevention_platform_haeun
02 AI 채용 평가	-	↗ github.com/haeun910/ai-recruit-trust
03 BARO	↗ baro-system.vercel.app	↗ github.com/haeun910/BARO
04 경남 보행안전	↗ haeun910.github.io/gyeongnam-walk-safety	↗ github.com/haeun910/gyeongnam-walk-safety
05 Moa	↗ moa-planner.vercel.app	↗ github.com/haeun910/moa